Assignment 1: A Scalar Autograd Engine and Neural Network API

MicroGrad from scratch — design, derivations and results

CSE485/CSE488 Deep Learning, Ain Shams University

October 21, 2026

Abstract

This report documents the design and verification of a scalar automatic differentiation engine (`src/engine.py`) and a small neural-network API (`src/nn.py`) implemented from scratch in Python, without PyTorch or any other autodiff library. We derive every local backward rule by hand, explain reverse-mode autodiff and the role of the topological sort, justify the implementation choices (closure-based backpropagation, set-based parent tracking, activation choice, weight initialisation), and present verification results: a hand-checked and finite-difference–checked test suite (**42/42 passing**), gradient checks that agree with central differences to a relative error of 5.8×10^{-10}, and a 337-parameter multilayer perceptron trained with hinge loss to **98.5%** accuracy on a two-moons classification task.

Contents

1 Overview and architecture **3**

2 The chain rule and reverse-mode autodiff **3**
2.1 Local derivatives (derived by hand) . . . 3

3 Topological sort and the backward pass **4**
3.1 Why a topological order is required . . . 4
3.2 Why post-order (append after the children loop) . . . 5
3.3 Worked example . . . 5

4 The diamond graph: `visited` and `+=` **5**

5 Design justifications **6**

6 The neural-network API **7**
6.1 Neuron, Layer, MLP . . . 7
6.2 Loss and optimisation . . . 7

7 Verification **8**
7.1 Gradient checking . . . 8
7.2 Test suite . . . 8

8 Experimental results **9**

9 Answers to the conceptual questions **10**

10 Reproducibility **11**

1 Overview and architecture

The assignment consists of two modules and their verification material:

File	Responsibility
`src/engine.py`	`Value` class: scalar data, graph edges, local backward rules, global `backward()`
`src/nn.py`	`Neuron`, `Layer`, `MLP`: parameterised composition of `Value` operations
`tests/`	pytest suites: template tests + extra property, edge-case and gradient-check tests
`demo.ipynb`	reproducible training run with plots
`report/`	this LaTeX report and its figures

A `Value` stores four things: the scalar `data`, the gradient `grad` ($\partial L/\partial v$ for the loss L that will later be differentiated), the set of parent nodes `_prev` from which it was computed, and the name of the operation `_op` that produced it. Each operation creates a new `Value` and attaches a closure `_backward` that implements its *local* contribution to the chain rule. The forward pass therefore builds a directed acyclic graph (DAG), and the backward pass walks it in reverse.

2 The chain rule and reverse-mode autodiff

Let L be a scalar loss computed by composing elementary functions,

$$L = f_k \circ f_{k-1} \circ \cdots \circ f_1(x_1, \ldots, x_n), \tag{1}$$

i.e. each intermediate value v_j depends on earlier values $v_j = f_j(v_{j_1}, \ldots)$. The multivariate chain rule states

$$\frac{\partial L}{\partial v_i} = \sum_{j : v_j \text{ uses } v_i} \frac{\partial L}{\partial v_j} \frac{\partial v_j}{\partial v_i}. \tag{2}$$

Equation (2) has a recursive structure that suggests two evaluation orders:

- **Forward-mode** differentiates as the forward pass proceeds, carrying one tangent per input. Cost: one sweep *per input direction*, i.e. $\mathcal{O}(n)$ sweeps for n inputs.
- **Reverse-mode** first evaluates the forward pass (recording the graph), then applies (2) *backwards*, starting from $\partial L/\partial L = 1$. Every node is visited once, so the total cost is $\mathcal{O}$(number of graph nodes) *regardless of how many inputs there are.*

This asymmetry is exactly why deep learning uses reverse-mode: a network has millions of parameters (inputs) but a single scalar loss (output). One backward sweep produces $\partial L/\partial p$ for *every* parameter p simultaneously. Gradient flow therefore runs from the output backwards to the inputs: the seed $\partial L/\partial L = 1$ is placed on the node from which we differentiate, and each node pushes its accumulated gradient to its own parents, scaled by its local derivative.

2.1 Local derivatives (derived by hand)

Each operator only needs its own partial derivatives; the engine multiplies them together via (2).

Addition. For $c = a + b$,

$$\frac{\partial c}{\partial a} = 1, \qquad \frac{\partial c}{\partial b} = 1 \quad \Longrightarrow \quad a.grad \mathrel{+}= c.grad, \;\; b.grad \mathrel{+}= c.grad. \tag{3}$$

Multiplication. For $c = a \cdot b$ (product rule):

$$\frac{\partial c}{\partial a} = b, \qquad \frac{\partial c}{\partial b} = a \quad \Longrightarrow \quad a.grad \mathrel{+}= b \cdot c.grad, \;\; b.grad \mathrel{+}= a \cdot c.grad. \tag{4}$$

Power. For $c = a^k$ with constant $k \in \mathbb{R}$:

$$\frac{\partial c}{\partial a} = k\, a^{k-1} \quad \Longrightarrow \quad a.grad \mathrel{+}= k\, a^{k-1} \cdot c.grad. \tag{5}$$

The exponent must be a constant: if k were itself a `Value`, the derivative would need the extra term $a^k \ln a$. The implementation asserts that the exponent is an `int` or `float`. The case $k = 0$ is special-cased (the result is the constant 1, derivative 0) to avoid evaluating a^{-1} when $a = 0$.

Tanh. With $\tanh x = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$,

$$\frac{\mathrm{d}}{\mathrm{d}x} \tanh x = \frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2} = \frac{4}{(e^x + e^{-x})^2} = 1 - \tanh^2 x. \tag{6}$$

Hence, for $t = \tanh a$:

$$a.grad \mathrel{+}= (1 - t^2) \cdot out.grad. \tag{7}$$

Note $|1 - t^2| \le 1$: the factor never amplifies the gradient, and for saturated outputs ($|t| \to 1$) it vanishes — the familiar vanishing-gradient behaviour of saturating activations.

Exponential. For $c = e^a$:

$$\frac{\partial c}{\partial a} = e^a = c \quad \Longrightarrow \quad a.grad \mathrel{+}= e^a \cdot c.grad. \tag{8}$$

Derived operators. Subtraction, negation and division are built from the primitives rather than given their own backward rules: $-a \equiv a \cdot (-1)$, $a - b \equiv a + (-b)$, $a/b \equiv a \cdot b^{-1}$, so $\partial(a/b)/\partial a = 1/b$ and $\partial(a/b)/\partial b = -a/b^2$ fall out of the multiplication and power rules automatically.

3 Topological sort and the backward pass

3.1 Why a topological order is required

Backpropagation may only visit a node after *every* node that consumes its gradient has finished contributing to it; otherwise the node would push a partial, still-incomplete gradient to its own parents. A topological order of the DAG is precisely the ordering in which every node appears after all of its dependencies; processing that list in **reverse** therefore starts at the output node and ends at the inputs, and when a node is reached its `grad` is final.

3.2 Why post-order (append after the children loop)

The DFS builds the list by appending a node *after* recursing into all of its children:

```
def build(node):
    if node not in visited:
        visited.add(node)
        for child in node._prev:
            build(child)
        topo.append(node)      # AFTER the loop -> post-order
```

Appending before the loop would place a node *before* its own children in `topo`, i.e. `reversed(topo)` would try to push gradient through a node before that node's own gradient had been assembled — silently wrong gradients.

3.3 Worked example

Take the graph used by the test suite, with $a = -2$, $b = 3$:

$$d = a \cdot b, \qquad e = a + b, \qquad f = d \cdot e. \tag{9}$$

A post-order DFS from f yields (one valid order) `[a, b, d, e, f]`; reversed, the backward pass proceeds `f, e, d, b, a`:

Node (reversed order)	Local contribution	`a.grad`	`b.grad`
f (seed)	$f.grad = 1$	0	0
e (upstream $= d \cdot f' = -6$)	$a.grad \mathrel{+}= 1 \cdot (-6)$; $b.grad \mathrel{+}= 1 \cdot (-6)$	-6	-6
d (upstream $= e \cdot f' = 1$)	$a.grad \mathrel{+}= b \cdot 1 = 3$; $b.grad \mathrel{+}= a \cdot 1 = -2$	$\mathbf{-3}$	$\mathbf{-8}$
b, a (leaves)	no outgoing edges	-3	-8

The final values $\partial f/\partial a = -3$ and $\partial f/\partial b = -8$ match the analytic derivatives of $f = ab(a+b) = a^2b + ab^2$: $\partial f/\partial a = 2ab + b^2 = -12 + 9 = -3$ and $\partial f/\partial b = a^2 + 2ab = 4 - 12 = -8$. This is exactly `tests/test_engine.py::test_chain_rule_and_backprop`.

4 The diamond graph: `visited` and `+=`

A diamond occurs when a node is consumed by several parents, e.g.

$$c = a + a, \qquad d = c \cdot a, \qquad a = 3. \tag{10}$$

Analytically $d = 2a^2$, so $\mathrm{d}d/\mathrm{d}a = 4a = 12$.

The `visited` set. Without it, the DFS would recurse into a once from c and again from d, appending a twice; the backward pass would then execute a's outgoing contribution twice, double-counting it, and in deep graphs the recursion would blow up combinatorially. With `visited`, each node appears exactly once.

Gradient accumulation with `+=`. Processing d first:

$$\begin{aligned} d.backward\text{-step:} \quad & c.grad \mathrel{+}= a \cdot 1 = 3, \qquad a.grad \mathrel{+}= c \cdot 1 = 6, \\ c.backward\text{-step (upstream 3):} \quad & a.grad \mathrel{+}= 1 \cdot 3 \text{ (from the first operand)}, \\ & a.grad \mathrel{+}= 1 \cdot 3 \text{ (from the second operand)}, \end{aligned}$$

giving $a.grad = 6 + 3 + 3 = 12 = 4a$. **If** the local rules used assignment ($a.grad = \dots$) instead of accumulation, the second operand's term would overwrite the first and $a.grad$ would end at 3 — a wrong gradient that no forward-value test would catch. The same property makes shared sub-expressions work in general: the accumulated gradient at a node is the *sum* over all consumption paths, which is exactly Eq. (2).

Both traps are pinned down by tests: `test_gradient_accumulation_diamond_graph` ($a + a$, expect 12), plus extra tests for $a \cdot a$, three-fold reuse, and shared sub-expressions.

5 Design justifications

Closure-based `_backward`. Each operation stores a closure capturing only its own operands, so a node carries everything needed to push gradient locally — no global dispatch table mapping operator names to gradient functions, and adding an operator (e.g. `relu`, `log`, `sigmoid`) means adding 5 lines in one place. Closures also keep the representation uniform: leaves simply keep the default no-op hook.

Upstream gradient passed as an argument. A classic subtlety: a closure of the form `def _backward(): ... out.grad ...` captures the node it is stored *on*, creating a reference cycle (`out -> out._backward -> out`). Such garbage can only be reclaimed by the cyclic garbage collector; with a training graph of ∼130 000 nodes per epoch this dominated runtime. Our closures therefore receive the upstream gradient explicitly, `node._backward(node.grad)`, and capture only *child* nodes — the graph stays acyclic and a finished epoch is freed instantly by reference counting. Measured on the demo workload (Table 1), this is a 2.2× speed-up; the remaining gap to `gc.disable()` is ordinary generational scanning of the still-live graph.

Table 1: Training-loop epoch cost (200 samples, `MLP(2,[16,16,1])`).

Implementation	ms / epoch
Closure capturing `out.grad` (cyclic)	1532
Upstream passed as argument (ours)	**669**
Reference: cyclic GC switched off	145

`_prev` as a `set`. Parents are stored in a set so that repeated uses of the same node (`a + a`) collapse to one edge: the graph models *distinct dependencies*, the DFS visits each node once, and the topology stays a DAG rather than a multigraph. Multiplicity of usage is carried by the gradients (multiple `+=` terms), not by duplicate edges.

Uniform weights in $[-1, 1]$ and bias. Small random weights keep the pre-activations of every layer inside the "linear" region of `tanh` at initialisation, so no unit starts saturated (where $1 - \tanh^2 \approx 0$ would kill its gradient), and the symmetric range keeps the expected pre-activation

centred at 0. Bias is drawn from the same range rather than fixed at 0 to break symmetry between units of a layer even before the first update. (Exact initialisation is unobservable through the public API, so the tests assert the documented range rather than specific values.)

Why `tanh`. `tanh` is zero-centred — outputs lie in $(-1, 1)$, so downstream pre-activations receive mixed-sign inputs and gradient signs do not oscillate as they can with sigmoid — while still being smooth and bounded, with a derivative expressible purely in terms of the forward output $(1 - t^2)$, i.e. it costs nothing beyond the forward pass. The final layer omits it (see below).

`backward()` does not zero gradients. Following PyTorch semantics, gradients accumulate across calls; the caller resets them with `reset_grad()`/`zero_grad()` between optimisation steps. This is documented in the docstring and covered by an explicit test.

6 The neural-network API

6.1 Neuron, Layer, MLP

A `Neuron` with n_{in} inputs computes

$$a = b + \sum_{i=1}^{n_{in}} w_i x_i, \qquad y = \tanh(a) \tag{11}$$

(or $y = a$ when `nonlin=False`). Its parameters are $\{w_1, \ldots, w_{n_{in}}, b\}$, i.e. $n_{in} + 1$ `Value`s. Locally, $\partial a/\partial w_i = x_i$ and $\partial a/\partial b = 1$, so Eq. (11)'s backward rule needs no new machinery.

A `Layer` applies n_{out} neurons to the same input vector and returns a single `Value` when $n_{out} = 1$ (so scalar heads compose naturally) or a list otherwise.

An `MLP` chains layers, `MLP(3,[4,4,1])`:

$$\mathbf{h}^{(1)} = \tanh(W_1\mathbf{x} + \mathbf{c}_1), \quad \mathbf{h}^{(2)} = \tanh(W_2\mathbf{h}^{(1)} + \mathbf{c}_2), \quad \hat{y} = W_3\mathbf{h}^{(2)} + c_3. \tag{12}$$

Hidden layers use `tanh`; the **last layer is linear**, so the network emits an unbounded score that the margin-based loss can push arbitrarily far — squashing the output would cap the achievable margin. Parameter count is the sum of layer sizes,

$$\sum_{\ell} \left(n_{in}^{(\ell)} + 1\right) n_{out}^{(\ell)} \;\Rightarrow\; (3{+}1)4 + (4{+}1)4 + (4{+}1)1 = 41 \quad \text{(and 337 for the demo model).} \tag{13}$$

`parameters()` flattens this hierarchy; `zero_grad()` resets each parameter's gradient between optimisation steps.

6.2 Loss and optimisation

For scores s_i and labels $y_i \in \{-1, +1\}$ the demo minimises the mean hinge (max-margin) loss with L2 regularisation:

$$L = \frac{1}{N} \sum_{i=1}^{N} \max\bigl(0,\; 1 - y_i s_i\bigr) + \lambda \sum_{p} p^2, \qquad \lambda = 10^{-4}. \tag{14}$$

Updates are full-batch SGD with exponential learning-rate decay,

$$p \leftarrow p - \eta_t \frac{\partial L}{\partial p}, \qquad \eta_{t+1} = 0.998\,\eta_t, \quad \eta_0 = 0.2. \tag{15}$$

The hinge term is implemented as `max(0, m) = m.relu()` built from the primitive operators — no new backward rules are required for the loss, the regulariser, or the optimiser; they are all just nodes in the graph.

7 Verification

7.1 Gradient checking

Analytic gradients are compared against central finite differences,

$$\frac{\partial L}{\partial p} \approx \frac{L(p+h) - L(p-h)}{2h}, \qquad h = 10^{-5} \text{ (notebook)}, \; h = 10^{-6} \text{ (tests)}, \tag{16}$$

which is second-order accurate in h. Three independent checks are used:

1. hand-derived gradients for every operator (`add, mul, sub, div`, `pow, tanh, exp, relu, log, sigmoid`), including edge cases: x^0, x^1, negative exponents, $x = 0$, $\exp(\pm 700)$, and overflow at $\exp(1000)$;
2. a **finite-difference check of *all* 41 parameters** of an `MLP(3,[4,4,1])` on a regression loss (`tests/test_nn_extra.py`);
3. a sampled parameter-wise check inside the notebook after training (Table 2).

Table 2: Post-training gradient check on the trained model (6 random parameters of 337).

Parameter index	Analytic	Numeric	Relative error
29	-1.9703×10^{-3}	-1.9703×10^{-3}	1.8×10^{-10}
145	-1.0000×10^{-3}	-1.0000×10^{-3}	2.5×10^{-10}
147	9.9958×10^{-4}	9.9958×10^{-4}	7.0×10^{-11}
218	1.7342×10^{-3}	1.7342×10^{-3}	2.0×10^{-10}
257	1.4557×10^{-3}	1.4557×10^{-3}	1.2×10^{-10}
289	1.4047×10^{-4}	1.4047×10^{-4}	5.8×10^{-10}
Maximum relative error			$\mathbf{5.8 \times 10^{-10}}$

7.2 Test suite

Table 3: Local `pytest` run (`PYTHONPATH=. pytest`); the course CI runs the same command.

Suite	Result
`tests/test_engine.py` (template)	9 / 9 passed
`tests/test_nn.py` (template)	3 / 3 passed
`tests/test_engine_extra.py` (added)	18 / 18 passed
`tests/test_nn_extra.py` (added)	12 / 12 passed
Total	**42 / 42 passed**

The added tests cover: topological-order property (every node strictly after its children, no duplicates), diamond and shared-sub-expression accumulation, scalars on the left of operators (`2*x`, `1-x`, `3/x`), numerical stability of `tanh`/`exp`/`sigmoid` at 0 and at $|x| \in \{500, 700, 1000\}$, parameter counts and `zero_grad()` hierarchy, single-output layer collapsing, the linear final layer, and a one-step SGD loss-decrease property test.

8 Experimental results

The demo notebook (`demo.ipynb`) is fully reproducible: seeds are fixed (`random`, `numpy`), the dataset is generated in-notebook, and it has been executed top-to-bottom with *Restart & Run All*. The task is a two-moons binary classification with 200 noisy samples ($\sigma = 0.15$), model `MLP(2,[16,16,1])` (337 parameters), hinge loss + L2, 200 SGD epochs.

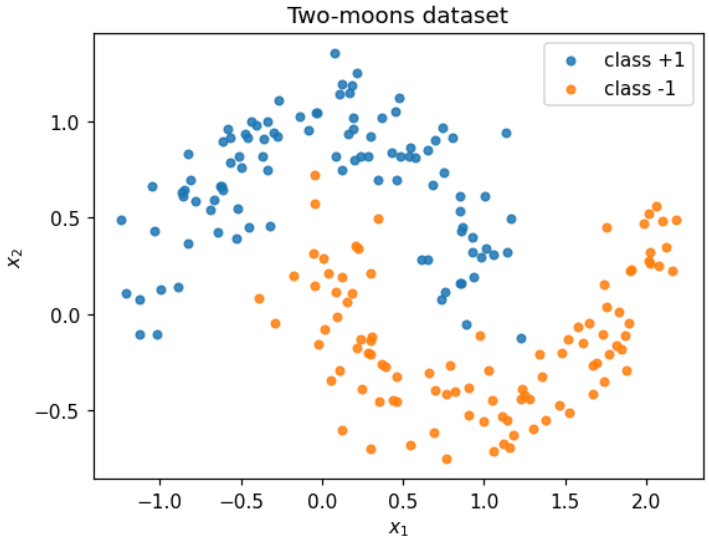


Figure 1: The two-moons dataset: 200 samples with Gaussian noise ($\sigma = 0.15$), labels in $\{-1, +1\}$.

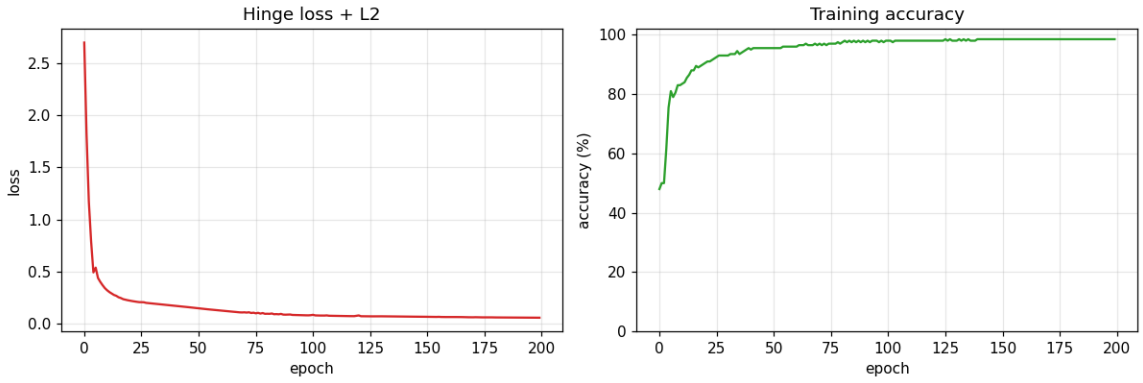


Figure 2: Left: mean hinge loss + L2 over training. Right: training accuracy. Both improve monotonically under full-batch SGD with learning-rate decay.

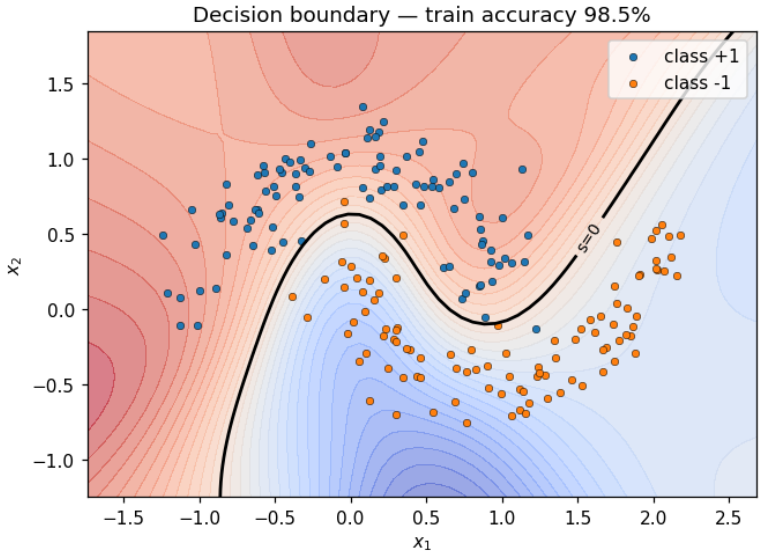


Figure 3: Decision boundary of the trained network (score $s(x) = 0$ contour) and the training points.

Loss falls monotonically from 2.70 to 0.0597 while training accuracy rises from 48% (random) to **98.5%** (Figure 2); the decision boundary in Figure 3 follows the two arcs closely, correctly separating the interleaved crescents. The loss keeps decreasing after accuracy saturates because the hinge loss continues to widen the margin on the few remaining ambiguous points.

9 Answers to the conceptual questions

The template repository does not list explicit questions; the set below covers the concepts the plan and assignment description emphasise (chain rule, reverse mode, topological sorting, diamonds, design choices, NN API). If the LMS publishes additional questions, they should be appended here.

1. **Why does the gradient flow from the output to the inputs?** Because L is a composition of local functions; Eq. (2) is a recursion from L downwards. Seeding $\partial L/\partial L = 1$ and visiting nodes in reverse topological order evaluates that recursion once, giving every parameter's gradient in a single $\mathcal{O}(|V|)$ sweep.

2. **Why is a topological sort needed, and why post-order?** A node may only be back-propagated through after all consumers of its gradient have contributed (otherwise it pushes an incomplete gradient). Post-order DFS lists every node after its dependencies; reversing that list yields a valid processing order starting from the output.

3. **Why the `visited` set?** DAGs share nodes. Without it, a shared node is visited once per path (exponential in depth) and would be appended multiple times to the topo list, causing its outgoing gradient to be applied several times — double-counted gradients. See Section 4.

4. **Why must local backward rules use `+=`?** Because a node can receive gradient from several parents (and from several operands of the same operation, as in $a + a$); Eq. (2) is a *sum* over paths. Assignment would let the last path overwrite the earlier ones: for $d = (a + a) \cdot a$ we would obtain 3 instead of 12.

5. **Why seed the root's gradient with** 1.0**?** We differentiate L with respect to itself, $\partial L/\partial L = 1$; everything else follows by the chain rule. It also means an intermediate node used as the seed yields the gradient of *that node* w.r.t. the inputs, which is what makes intermediate-value tests possible.

6. **What would change for a vector-valued output?** Reverse-mode gives one output's gradient per sweep; a Jacobian with m outputs requires m backward passes (or a framework with vector-Jacobian products). Deep learning keeps a scalar loss precisely so one pass suffices.

7. **Why is `pow` restricted to constant exponents?** $\partial_a a^k = ka^{k-1}$ holds only for constant k; a `Value` exponent would add $a^k \ln a \cdot \partial k/\partial a$, which the local rule does not implement. An assertion fails fast instead of returning a silently wrong gradient.

8. **Why uniform weights in** $[-1, 1]$**?** Keeps initial pre-activations in `tanh`'s linear region ($1 - \tanh^2 \approx 1$) so every unit can learn from step 1; symmetric random values also break unit symmetry. See Section 5.

9. **Why `tanh` instead of sigmoid?** Zero-centred outputs ($(-1, 1)$) give better-conditioned gradients for downstream layers, while remaining smooth and bounded; the derivative $1 - t^2$ reuses the forward value. The output layer stays linear so scores are not clipped by the margin loss.

10. **How does this compare to PyTorch?** PyTorch executes the same two-phase protocol (record graph in forward, walk it in reverse) but on tensors with fused C++/CUDA kernels, a dynamic autograd graph per iteration, and `Tensor.grad` accumulation that is semantically identical to our `+=`. Our `Value` is the scalar special case of a `Tensor`.

11. **Why is NumPy/PyTorch not allowed here?** The point of the exercise is to internalise the mechanics — computational graphs, local derivatives, topological ordering, accumulation — rather than to obtain gradients from a library. NumPy is used in the notebook only for dataset generation and plotting, never inside `engine.py` / `nn.py`.

10 Reproducibility

```
python -m pytest            # 42 tests, from the repository root
jupyter nbconvert --to notebook --execute demo.ipynb   # or Restart & Run All
cd report && tectonic main.tex                          # this PDF
```

No PyTorch (or any autodiff library) is imported by `src/engine.py` or `src/nn.py`; the engine depends only on the standard-library `math` module.